

Fiche de révision — Data Science (niveau alternant)

Tout ce que tu dois savoir pour tes entretiens techniques, expliqué simplement

Comment lire cette fiche. Chaque notion est expliquée en 3 temps : 🧑 *l'idée simple* (comme à un enfant), 🧠 *les mots exacts à connaître*, et quand c'est utile 💻 *le code* + 💡 *ce qu'on peut te demander en entretien*. Lis sans te presser, et reviens-y plusieurs fois. Tu n'as pas besoin de tout retenir par cœur : il faut **comprendre** et savoir **expliquer avec tes mots**.

SOMMAIRE

1. C'est quoi la data science ? (le vocabulaire de base)
 2. Les types de données
 3. Python pour la data (NumPy, Pandas, visualisation)
 4. SQL (parler aux bases de données)
 5. Statistiques essentielles
 6. Le processus d'un projet data (de A à Z)
 7. Préparer les données (nettoyage & transformation)
 8. Le Machine Learning en détail
 9. Le Deep Learning (notions)
 10. NLP, LLM & IA générative
 11. MLOps (mettre un modèle en production)
 12. Glossaire express
 13. Questions d'entretien technique (avec réponses)
 14. Plan de révision (checklist)
-

1. C'est quoi la data science ? (le vocabulaire de base)

🧑 **L'idée simple.** Imagine une montagne de tickets de caisse d'un supermarché. Tout seuls, ces tickets ne disent rien. La data science, c'est l'art de **lire cette montagne** pour répondre à des questions utiles : « Qu'est-ce que les gens achètent le dimanche ? », « Vais-je manquer de

lait la semaine prochaine ? ». On utilise des **données** pour **comprendre** (le passé) et **prédire** (le futur).

Les mots exacts.

- **Donnée (data)** : une information enregistrée (un âge, un prix, une photo, un texte).
- **Dataset (jeu de données)** : un tableau de données. C'est comme un grand tableau Excel.
- **Ligne / observation / individu** : une « fiche ». Ex. *un* client, *une* vente.
- **Colonne / variable / feature (caractéristique)** : une information mesurée sur chaque ligne. Ex. la colonne « âge », la colonne « prix ».
- **Target (cible / label)** : la colonne qu'on veut **prédire**. Ex. « le client va-t-il partir ? oui/non ».
- **Features (X) vs Target (y)** : les **X** sont les indices qu'on a, le **y** est la réponse à deviner.
Image : les X sont les symptômes, le y est la maladie.

	colonne « âge » (feature)	colonne « revenu » (feature)	colonne « achète ? » (target)
client 1 (ligne)	25	1800	oui
client 2 (ligne)	40	3200	non

 **Entretien** : « Quelle différence entre une feature et la target ? » → *Les features sont les variables d'entrée qui servent à prédire ; la target est la variable de sortie qu'on cherche à deviner.*

Data analyst vs data scientist (question classique) :

- Le **data analyst** répond à des questions métier existantes : il explore, analyse, fait des tableaux de bord (« pourquoi les ventes baissent ? »).
- Le **data scientist** va plus loin vers la **prédiction** avec du machine learning (« quels clients vont partir le mois prochain ? »).
- Dans la vraie vie, les deux se recouvrent beaucoup.

2. Les types de données

 **L'idée simple.** Les données ont des « familles ». On ne traite pas un nombre comme on traite un mot.

Les familles de variables :

- **Numérique (quantitative)** : des nombres.
 - *Continue* : peut prendre une infinité de valeurs (taille = 1,73 m ; prix = 12,49 €).
 - *Discrète* : des nombres entiers comptables (nombre d'enfants = 0, 1, 2...).
- **Catégorielle (qualitative)** : des catégories, pas des quantités.
 - *Nominale* : pas d'ordre (couleur : rouge/bleu/vert ; ville).
 - *Ordinale* : un ordre existe (satisfaction : faible < moyen < élevé).
- **Temporelle (date/heure)** : une date, un horodatage (série temporelle = données dans le temps).
- **Texte (non structuré)** : des phrases, des avis clients.
- **Image / son / vidéo** : données non structurées aussi.

 **Entretien** : sois capable de dire, pour une colonne donnée, si elle est numérique ou catégorielle — car **le type décide du traitement** (un modèle ne « mange » que des nombres, donc le texte/les catégories devront être **encodés**, voir §7).

3. Python pour la data

Python est le langage roi de la data. Voici l'essentiel.

3.1 Rappels Python de base (très court)

Variables et types

```
age = 25          # int (entier)
```

```
prix = 12.5       # float (décimal)
```

```
nom = "Lucien"    # str (texte)
```

```
actif = True      # bool (vrai/faux)
```

Liste : une suite ordonnée d'éléments

```
fruits = ["pomme", "banane", "kiwi"]
```

```
fruits.append("orange") # ajoute à la fin
```

```
print(fruits[0])      # "pomme" (on compte à partir de 0 !)
```

Dictionnaire : des paires clé -> valeur (comme un annuaire)

```
client = {"nom": "Lucien", "age": 25}
```

```
print(client["age"]) # 25
```

Boucle : répéter une action

```
for f in fruits:
```

```
    print(f)
```

Condition

```
if age >= 18:
```

```
    print("majeur")
```

```
else:
```

```
    print("mineur")
```

Fonction : une recette réutilisable

```
def double(x):
```

```
    return x * 2
```

```
print(double(2)) # 4
```

🧑 Une liste = une file d'attente d'objets. Un dictionnaire = un annuaire (tu donnes un nom, il te rend un numéro).

3.2 NumPy — calculer vite sur des tableaux de nombres

🧑 **L'idée simple.** NumPy, c'est une **super calculatrice** qui applique une opération à des milliers de nombres d'un coup.

```
import numpy as np
```

```
a = np.array([1, 2, 3, 4])
```

```
print(a * 2)    # [2 4 6 8] -> multiplie TOUT d'un coup
```

```
print(a.mean()) # 2.5 (moyenne)
```

```
print(a.sum())  # 10
```

```
print(a[a > 2]) # [3 4] -> filtre les valeurs > 2
```

🧠 **Array (tableau)** : structure de NumPy, plus rapide qu'une liste pour les calculs. On parle aussi de **vecteur** (1 dimension) et de **matrice** (2 dimensions).

3.3 Pandas — LE couteau suisse de la data

🧑 **L'idée simple.** Pandas, c'est **Excel piloté par du code**. On manipule des tableaux appelés **DataFrame**.

🧠 **DataFrame** = un tableau (lignes + colonnes). **Series** = une seule colonne.

```
import pandas as pd
```

```
# 1) Charger des données
```

```
df = pd.read_csv("ventes.csv") # depuis un fichier CSV
```

```
# df = pd.read_excel("ventes.xlsx")
```

```
# 2) Regarder les données (réflexes à TOUJOURS avoir)
```

```
df.head()    # les 5 premières lignes
```

```
df.shape     # (nb_lignes, nb_colonnes)
```

```
df.info()    # types des colonnes + valeurs manquantes
```

```
df.describe() # stats (moyenne, min, max...) des colonnes numériques
```

```
# 3) Sélectionner
```

```
df["prix"]    # une colonne (Series)
```

```
df[["prix", "ville"]] # plusieurs colonnes
```

```
df.loc[0]     # la ligne d'index 0
```

```
df.iloc[0:5]          # les 5 premières lignes (par position)
```

4) Filtrer (très important)

```
df[df["prix"] > 100]    # lignes où prix > 100
```

```
df[(df["prix"] > 100) & (df["ville"] == "Marseille")] # & = ET, | = OU
```

5) Créer une colonne

```
df["prix_ttc"] = df["prix"] * 1.2
```

6) Valeurs manquantes

```
df.isna().sum()        # combien de trous par colonne
```

```
df = df.dropna()       # supprimer les lignes avec trous
```

```
df["prix"] = df["prix"].fillna(df["prix"].median()) # remplir par la médiane
```

7) Grouper et agréger (= "tableau croisé dynamique")

```
df.groupby("ville")["prix"].mean()    # prix moyen par ville
```

```
df.groupby("ville")["prix"].agg(["mean", "sum", "count"])
```

8) Trier

```
df.sort_values("prix", ascending=False).head(5) # top 5 prix
```

9) Fusionner deux tables (comme un JOIN SQL)

```
result = pd.merge(commandes, clients, on="client_id", how="left")
```

Entretien (très fréquent) :

- « Comment gères-tu les valeurs manquantes ? » → Je les quantifie (`isna().sum()`), je comprends pourquoi elles manquent, puis selon le cas je supprime (`dropna`) si c'est marginal ou j'impute (`fillna` par moyenne/médiane/mode). Ça dépend du contexte et de l'impact métier.
- « C'est quoi un groupby ? » → Regrouper les lignes par une catégorie puis calculer une statistique par groupe (somme, moyenne...). Comme un tableau croisé dynamique.

3.4 Visualisation (Matplotlib / Seaborn)

🧑 **L'idée simple.** Un bon graphique vaut mille tableaux de chiffres.

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
sns.histplot(df["prix"])    # distribution d'une variable
```

```
sns.boxplot(x="ville", y="prix", data=df) # comparer + voir les outliers
```

```
sns.scatterplot(x="age", y="revenu", data=df) # relation entre 2 variables
```

```
sns.heatmap(df.corr(), annot=True) # matrice de corrélation
```

```
plt.show()
```

🧠 **Quel graphique pour quoi ?**

- **Histogramme** : voir la répartition d'une variable numérique.
- **Boxplot (boîte à moustaches)** : comparer des groupes + repérer les **valeurs aberrantes (outliers)**.
- **Scatter plot (nuage de points)** : voir la **relation entre deux** variables numériques.
- **Bar plot** : comparer des catégories.
- **Courbe (line)** : une évolution dans le temps.

💡 **Entretien** : « Tu veux comparer le salaire selon le métier, quel graphe ? » → *Un boxplot (ou barplot des moyennes)*.

4. SQL — parler aux bases de données

🧑 **L'idée simple.** Une base de données, c'est un **classeur** avec plusieurs **tables** (feuilles). SQL est la **langue** pour demander : « donne-moi les clients de Marseille qui ont dépensé plus de 100 € ». Tu poses une question, la base te répond avec un tableau.

🧠 **Mots de base** : une **table** (tableau), une **ligne** (un enregistrement), une **colonne** (un champ), une **clé primaire** (identifiant unique d'une ligne, ex. **client_id**), une **clé étrangère** (qui pointe vers la clé primaire d'une autre table, pour relier les tables).

4.1 La requête de base : SELECT

SELECT nom, ville -- quelles colonnes je veux

FROM clients -- dans quelle table

WHERE ville = 'Marseille' -- quelles lignes (filtre)

ORDER BY nom ASC -- trier (ASC = croissant, DESC = décroissant)

LIMIT 10; -- n'en garder que 10

👉 Ordre de lecture mental : **FROM** (la table) → **WHERE** (je filtre) → **SELECT** (je choisis les colonnes) → **ORDER BY** (je trie) → **LIMIT**.

SELECT * veut dire « toutes les colonnes ».

4.2 Agréger : GROUP BY + fonctions

👤 On regroupe puis on calcule un chiffre par groupe.

SELECT ville, COUNT(*) AS nb_clients, AVG(montant) AS panier_moyen

FROM commandes

GROUP BY ville

HAVING AVG(montant) > 50 -- filtre APRÈS le regroupement

ORDER BY nb_clients DESC;

🧠 **Fonctions d'agrégation** : COUNT() (compter), SUM() (somme), AVG() (moyenne), MIN(), MAX().

⚠️ **WHERE vs HAVING** (piège d'entretien archi-classique) :

- WHERE filtre les lignes **AVANT** le regroupement.
- HAVING filtre **APRÈS** le GROUP BY, sur le résultat agrégé.

4.3 Relier des tables : les JOIN

🧠 **L'idée simple.** Tes infos sont dans 2 classeurs : un avec les commandes, un avec les clients. Le **JOIN** les recolle grâce à une colonne commune (**client_id**).

```
SELECT c.nom, o.montant
```

```
FROM commandes o
```

```
JOIN clients c ON o.client_id = c.client_id;
```

🧠 **Types de JOIN :**

- **INNER JOIN** : garde seulement les lignes qui matchent dans les **deux** tables.
- **LEFT JOIN** : garde **toutes** les lignes de la table de gauche, même sans correspondance à droite (les manquants seront **NULL**).
- **RIGHT JOIN** : l'inverse.
- **FULL JOIN** : tout des deux côtés.

Image : INNER = l'intersection de deux cercles ; LEFT = tout le cercle de gauche.

4.4 Le top N par catégorie : fonctions fenêtre

🧠 « Donne-moi le client n°1 de chaque ville. »

```
SELECT *
```

```
FROM (
```

```
    SELECT nom, ville, montant,
```

```
           ROW_NUMBER() OVER (PARTITION BY ville ORDER BY montant DESC) AS rang
```

```
FROM clients
```

```
) t
```

```
WHERE rang = 1;
```

🧠 **Fonctions fenêtre :** **ROW_NUMBER()**, **RANK()**, **DENSE_RANK()** numérotent les lignes à l'intérieur d'un groupe (**PARTITION BY**) sans fusionner les lignes (contrairement à **GROUP BY**).

4.5 Sous-requête

SELECT nom FROM clients

WHERE client_id IN (SELECT client_id FROM commandes WHERE montant > 1000);

🧑 Une requête à l'intérieur d'une autre : d'abord la base trouve les gros acheteurs, puis leurs noms.

💡 Questions SQL fréquentes en entretien :

- Écris une requête « top 5 clients par chiffre d'affaires » → `SELECT client_id, SUM(montant) AS ca FROM commandes GROUP BY client_id ORDER BY ca DESC LIMIT 5;`
 - Différence WHERE / HAVING ? (voir ci-dessus)
 - INNER vs LEFT JOIN ?
 - Compter les valeurs distinctes → `SELECT COUNT(DISTINCT client_id) FROM commandes;`
 - **Pense à voix haute** pendant que tu écris : ça rassure le recruteur.
-

5. Statistiques essentielles

🧑 **L'idée simple.** Les stats, c'est **résumer** une montagne de chiffres avec quelques nombres simples, et **mesurer la confiance** qu'on peut leur accorder.

5.1 Résumer une variable (statistiques descriptives)

- **Moyenne (mean)** : on additionne tout et on divise par le nombre. *Sensible aux valeurs extrêmes.*
- **Médiane** : la valeur du milieu quand on trie. *Robuste aux extrêmes.* (Ex. salaires : la médiane est plus honnête que la moyenne car un PDG tire la moyenne vers le haut.)
- **Mode** : la valeur la plus fréquente.
- **Écart-type (standard deviation)** : mesure à quel point les valeurs sont **dispersées** autour de la moyenne. Petit = valeurs serrées ; grand = valeurs éparpillées.
- **Variance** : l'écart-type au carré (même idée).
- **Min / Max / Quartiles (Q1, Q2=médiane, Q3)** : Q1 = 25 % des données sont en dessous ; Q3 = 75 %. **IQR = Q3 - Q1** sert à repérer les outliers.

🧑 *Moyenne = le centre ; écart-type = à quel point ça part dans tous les sens.*

5.2 Distribution

- **Distribution** : la forme de la répartition des valeurs.
- **Distribution normale (courbe en cloche)** : symétrique, la plupart des valeurs au centre. Beaucoup de méthodes la supposent.
- **Asymétrie (skew)** : la courbe penche d'un côté (ex. revenus : longue traîne à droite).

5.3 Relations entre variables

- **Corrélation** : deux variables évoluent-elles ensemble ? Coefficient entre **-1 et +1**. +1 = montent ensemble ; -1 = l'une monte quand l'autre descend ; 0 = aucun lien linéaire.
-  **Corrélation ≠ causalité**. Les ventes de glaces et les noyades montent ensemble en été... mais l'une ne cause pas l'autre (c'est la chaleur, la **variable cachée**). Pour prouver une cause, il faut une **expérimentation** (ex. test A/B).

5.4 Tester une idée (notions)

- **Test d'hypothèse** : on teste « est-ce un vrai effet ou juste du hasard ? ».
- **p-value** : la probabilité d'observer ce résultat si l'effet n'existait pas. **$p < 0,05$** → on considère l'effet « significatif » (peu probable d'être dû au hasard).
- **Test A/B** : montrer la version A à un groupe, la version B à un autre, et comparer (très utilisé en entreprise pour décider).

Entretien :

- « Moyenne ou médiane pour des salaires ? » → *La médiane, car la moyenne est tirée par les très hauts salaires.*
- « Corrélation = causalité ? » → *Non. Il peut y avoir une variable cachée ou une coïncidence ; il faut une expérimentation pour parler de cause.*
- « Une p-value de 0,03, ça veut dire quoi ? » → *Il n'y a que 3 % de chances d'observer ce résultat par pur hasard → effet probablement réel (au seuil 5 %).*

6. Le processus d'un projet data (de A à Z)

 **L'idée simple.** Un projet data, c'est une **recette de cuisine** avec des étapes dans l'ordre. On appelle ça souvent **CRISP-DM**.

Les étapes :

1. **Comprendre le besoin métier** : quelle est la vraie question ? Quel sera le succès ? *(L'étape la plus importante et souvent oubliée.)*
2. **Comprendre & collecter les données** : où sont-elles ? sont-elles fiables ?
3. **Préparer / nettoyer les données** : 70-80 % du temps réel ! (valeurs manquantes, doublons, formats...).
4. **Explorer (EDA = Exploratory Data Analysis)** : graphiques, stats, corrélations pour « sentir » les données.
5. **Modéliser** : choisir et entraîner un modèle (si on fait du ML).
6. **Évaluer** : le modèle est-il bon ? (avec les bonnes métriques, §8).
7. **Déployer** : mettre le modèle/dashboard à disposition des utilisateurs.
8. **Surveiller (monitoring)** : le modèle reste-t-il bon dans le temps ? (§11).

💡 **Entretien — la mise en situation type** : « Les ventes baissent, explique-moi pourquoi. »
Réponse en or : **1)** je cadre (quelle période ? quel périmètre ? définition de "baisse" ?), **2)** je vérifie la qualité des données, **3)** je segmente (par produit, région, canal, dans le temps) pour localiser la baisse, **4)** je formule et teste des hypothèses, **5)** je restitue clairement avec des recommandations actionnables — en distinguant corrélation et piste causale. ➡ **Le réflexe gagnant : cadrer d'abord, ne pas se jeter sur un modèle.**

7. Préparer les données (nettoyage & transformation)

🧑 **L'idée simple.** Avant de cuisiner, on **lave et on épluche** les légumes. Les données brutes sont sales : trous, doublons, fautes, formats mélangés.

🧠 **Les opérations clés :**

- **Valeurs manquantes (NaN)** : supprimer (`dropna`) ou imputer (`fillna` par moyenne/médiane/mode, ou méthode plus fine).
- **Doublons** : `df.drop_duplicates()`.
- **Outliers (valeurs aberrantes)** : un âge de 200 ans. On les détecte (boxplot, IQR) puis on décide : corriger, supprimer, ou garder.
- **Encodage des catégories** (un modèle ne lit que des nombres !) :
 - **One-Hot Encoding** : une colonne « couleur » (rouge/bleu) devient 2 colonnes 0/1. *Pour les catégories sans ordre.*
 - **Label Encoding** : rouge→0, bleu→1. *Pour les catégories ordonnées ou les arbres.*
- **Mise à l'échelle (scaling)** des variables numériques (utile pour KNN, régressions, réseaux de neurones — pas pour les arbres) :

- **Normalisation (Min-Max)** : ramène tout entre 0 et 1.
- **Standardisation (Z-score)** : moyenne 0, écart-type 1.
- *Pourquoi ? Pour que « salaire » (en milliers) et « âge » (en dizaines) pèsent autant.*
- **Feature engineering** : créer des variables utiles (ex. à partir d'une date, créer « jour de la semaine », « mois »). *Souvent ce qui fait la différence d'un bon modèle.*

⚠ **Train/Test split AVANT de transformer** : on sépare d'abord, puis on calcule les paramètres de transformation (moyenne, etc.) **sur le train seulement**, pour éviter la **fuite de données (data leakage)** — sinon le modèle « triche » en voyant des infos du test.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

💡 **Entretien** : « Pourquoi standardiser ? » → *Pour mettre les variables à la même échelle, sinon celles aux grands nombres dominant. Inutile pour les arbres de décision.*

8. Le Machine Learning en détail

🧑 **L'idée simple.** Le machine learning (apprentissage automatique), c'est apprendre à un ordinateur à **trouver des règles tout seul** à partir d'exemples, au lieu de lui écrire les règles à la main. *Comme un enfant qui apprend à reconnaître un chat en voyant beaucoup de chats, sans qu'on lui donne la définition exacte d'un chat.*

8.1 Les grandes familles

- **Apprentissage supervisé** : on a des exemples **avec la réponse** (données étiquetées). Le modèle apprend à prédire la réponse.
 - **Régression** : prédire un **nombre** (prix d'une maison, température).
 - **Classification** : prédire une **catégorie** (spam/non-spam, malade/sain).
- **Apprentissage non supervisé** : **pas de réponse** donnée. Le modèle trouve des structures cachées.
 - **Clustering** : regrouper des éléments similaires (segmenter des clients).
 - **Réduction de dimension** : résumer beaucoup de colonnes en quelques-unes (PCA/ACP).
- **Apprentissage par renforcement** : un agent apprend par **essais/erreurs** avec des récompenses (jeux, robots). *Moins courant en alternance data classique.*

💡 *Image : supervisé = réviser avec le corrigé ; non supervisé = ranger une boîte de Lego par ressemblance sans notice.*

8.2 Les algorithmes à connaître (expliqués simplement)

- **Régression linéaire** : trace la **meilleure droite** à travers les points pour prédire un nombre. $\text{prix} = a \times \text{surface} + b$.
- **Régression logistique** : malgré son nom, c'est de la **classification** (oui/non). Elle sort une **probabilité** entre 0 et 1 (ex. 0,8 → 80 % de chances que ce soit un faux billet).
- **K-Nearest Neighbors (KNN)** : « dis-moi qui sont tes voisins, je te dirai qui tu es ». On classe un point selon la majorité de ses **k plus proches voisins**.
- **Arbre de décision** : une suite de questions oui/non (« âge > 30 ? revenu > 2000 ? ») qui aboutit à une décision. Très **lisible**.
- **Random Forest (forêt aléatoire)** : **beaucoup d'arbres** qui votent ensemble → plus robuste qu'un seul arbre. (Famille « ensemble ».)
- **Gradient Boosting (XGBoost, LightGBM)** : des arbres construits l'un après l'autre, chacun corrigeant les erreurs du précédent. **Très performant** sur données tabulaires (souvent gagnant en compétition).
- **K-Means** (non supervisé) : regroupe les points en **k clusters** en cherchant des centres. *Tu choisis k.*
- **PCA / ACP (Analyse en Composantes Principales)** (non supervisé) : **compresse** plusieurs colonnes corrélées en quelques « super-colonnes » qui gardent le maximum d'information. Utile pour visualiser ou accélérer.

👤 *Régression logistique = un juge qui donne un pourcentage de culpabilité. Random Forest = un jury de plein d'experts qui votent.*

8.3 Le grand ennemi : surapprentissage (overfitting)

- **Overfitting (surapprentissage)** : le modèle **apprend par cœur** les données d'entraînement, **bruit compris**, et se plante sur de nouvelles données. *Comme un élève qui mémorise les corrigés sans comprendre.*
- **Underfitting (sous-apprentissage)** : le modèle est **trop simple** et rate même les exemples d'entraînement.
- **Compromis biais-variance** : trop simple = beaucoup de **biais** (underfit) ; trop complexe = beaucoup de **variance** (overfit). Le but : le juste milieu.
- **Comment éviter l'overfitting ?** plus de données, modèle plus simple, **régularisation** (L1/L2), **validation croisée**, arrêter l'entraînement au bon moment.

8.4 Bien évaluer : train / validation / test

- **Train (entraînement)** : pour apprendre.
- **Validation** : pour régler les **hyperparamètres** et comparer les modèles.

- **Test** : gardé de côté **jusqu'à la fin** pour estimer la performance réelle sur des données jamais vues.
- **Validation croisée (cross-validation)** : on découpe en k morceaux, on entraîne sur k-1 et on teste sur le dernier, k fois. → estimation plus fiable.
- **Hyperparamètres** : les réglages qu'on choisit **avant** l'entraînement (ex. le **k** de KNN, la profondeur d'un arbre). On les optimise avec **GridSearchCV**.

8.5 Les métriques (TRÈS demandé en entretien)

Pour la régression (prédire un nombre) :

- **MAE** (erreur absolue moyenne) : l'erreur moyenne en valeur absolue. Simple à interpréter.
- **RMSE** : pénalise davantage les **grosses** erreurs (racine de la moyenne des erreurs au carré).
- **R²** : entre 0 et 1, la part de la variation expliquée par le modèle (1 = parfait).

Pour la classification (prédire une catégorie) — via la matrice de confusion :

	Prédit Positif	Prédit Négatif
Réel Positif	Vrai Positif (VP)	Faux Négatif (FN)
Réel Négatif	Faux Positif (FP)	Vrai Négatif (VN)

- **Accuracy (exactitude)** = $(VP + VN) / \text{total}$. ⚠ **Trompeuse si les classes sont déséquilibrées** (si 99 % des mails sont normaux, prédire « jamais spam » donne 99 % d'accuracy mais est inutile).
- **Precision (précision)** = $VP / (VP + FP)$: quand le modèle dit « positif », a-t-il raison ? *Importante quand un faux positif coûte cher (ex. accuser à tort).*
- **Recall (rappel/sensibilité)** = $VP / (VP + FN)$: parmi les vrais positifs, combien en a-t-on attrapés ? *Importante quand rater un positif coûte cher (ex. détecter une maladie).*
- **F1-score** : moyenne équilibrée de precision et recall (utile si classes déséquilibrées).
- **Courbe ROC / AUC** : mesure la capacité à séparer les classes (AUC = 1 parfait, 0,5 = hasard).

💡 **Entretien — LA question piège** : « Pour détecter une fraude (rare), tu regardes quelle métrique ? » → *Pas l'accuracy (trop de "non-fraude") ; je regarde le recall (ne pas rater de fraude) et la precision, équilibrés par le F1, et l'AUC.*

8.6 Workflow scikit-learn complet (à savoir réciter)

```
from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import classification_report, confusion_matrix

# 1) Séparer features (X) et target (y)

X = df.drop("fraude", axis=1)

y = df["fraude"]

# 2) Train / test

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42, stratify=y)

# 3) Mise à l'échelle (apprise sur le train uniquement)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train) # fit + transform sur le train

X_test = scaler.transform(X_test)      # transform seulement sur le test

# 4) Entraîner le modèle

model = LogisticRegression()

model.fit(X_train, y_train)

# 5) Prédire

y_pred = model.predict(X_test)

# 6) Évaluer
```



```
print(confusion_matrix(y_test, y_pred))
```

```
print(classification_report(y_test, y_pred)) # precision, recall, f1
```

🧠 **Réflexe clé** : **fit** = apprendre ; **transform** = appliquer ; **predict** = prédire. On **fit sur le train, jamais sur le test**.

9. Le Deep Learning (notions)

🧑 **L'idée simple**. Le deep learning, ce sont des **réseaux de neurones** : on empile des couches de petits calculateurs qui apprennent des motifs de plus en plus complexes. *Très fort pour les images, le son et le texte ; souvent inutilement lourd pour un simple tableau Excel (un Random Forest suffit).*

🧠 **Vocabulaire** :

- **Neurone** : une mini-fonction qui prend des entrées, les pondère (**poids**), et sort un nombre.
- **Couches (layers)** : entrée → couches cachées → sortie. « Profond » = beaucoup de couches.
- **Poids (weights)** : les valeurs que le réseau ajuste pour apprendre.
- **Fonction d'activation** (ReLU, sigmoïde) : ajoute de la « non-linéarité » pour apprendre des choses complexes.
- **Rétropropagation (backpropagation)** : la méthode pour corriger les poids en fonction des erreurs.
- **Epoch** : un passage complet sur toutes les données d'entraînement.
- **Architectures** :
 - **CNN** (réseau convolutif) → **images** (reconnaissance visuelle).
 - **RNN / LSTM** → **séquences** (texte, séries temporelles).
 - **Transformer** → la base des **LLM** modernes (voir §10).

💡 **Entretien** : « Deep learning ou Random Forest pour un tableau client ? » → *Pour des données tabulaires, je commence par un modèle simple/arbres (souvent meilleur et plus rapide). Le deep learning brille surtout sur images, son et texte.*

10. NLP, LLM & IA générative

🧠 **L'idée simple.** Le NLP (traitement du langage naturel) apprend aux machines à **comprendre et produire du texte**. Un **LLM** (grand modèle de langage, comme ChatGPT ou Claude) est un programme géant qui, à la base, fait une seule chose : **deviner le mot suivant** le plus probable... mais tellement bien qu'il sait résumer, traduire, coder, répondre.

🧠 **Vocabulaire essentiel :**

- **Token** : un morceau de texte (souvent un bout de mot). Le modèle lit et écrit token par token.
- **Embedding** : transformer un mot/texte en **vecteur de nombres** qui capture son **sens**. Deux mots proches en sens ont des vecteurs proches (« roi » et « reine »).
- **Transformer** : l'architecture (2017) derrière les LLM ; son mécanisme d'**attention** lui permet de regarder tout le contexte d'une phrase à la fois.
- **Pré-entraînement** : le modèle apprend sur d'énormes quantités de texte.
- **Fine-tuning** : ré-entraîner légèrement un modèle existant sur **tes** données pour une tâche précise.
- **Prompt** : la consigne que tu donnes au modèle. **Prompt engineering** = l'art de bien formuler.
- **RAG (Retrieval-Augmented Generation)** : on **branche** le LLM sur une base de documents : il **va chercher** les infos pertinentes puis répond en s'appuyant dessus. *Très utilisé en entreprise pour répondre sur des documents internes sans tout réentraîner — et sans exposer les données.*
- **Hallucination** : quand le LLM invente une réponse fausse avec assurance. **Limite majeure** → toujours vérifier.
- **Température** : réglage du « niveau de créativité/aléa » des réponses.

💡 **Entretien (très 2026)** : « Que penses-tu des LLM dans ton futur métier ? » → *Un accélérateur formidable (aide au code, exploration, synthèses), mais à utiliser avec esprit critique : un LLM peut halluciner et n'est pas un data engineer. Je m'en sers comme copilote, en gardant la vérification et le sens métier. Pour des données sensibles, des approches comme le RAG ou un LLM hébergé en local protègent la confidentialité.*

11. MLOps — mettre un modèle en production (notions)

🧑 L'idée simple. Faire un bon modèle dans un notebook, c'est 20 % du travail. Le **MLOps**, c'est tout ce qui permet de **faire tourner ce modèle pour de vrai, durablement** : le déployer, le surveiller, le mettre à jour.

🧠 **Notions :**

- **Mise en production (déploiement)** : rendre le modèle utilisable, souvent via une **API** (le modèle répond à des requêtes).
- **Pipeline** : la chaîne automatisée données → préparation → modèle → prédiction.
- **Versioning** : garder l'historique du **code** (Git) ET des **données/modèles**.
- **CI/CD** : automatiser les tests et le déploiement.
- **Conteneurisation (Docker)** : emballer le modèle + son environnement pour qu'il tourne partout pareil.
- **Monitoring** : surveiller que le modèle reste bon dans le temps.
- **Data drift / model drift** : quand le monde change, les données changent, et le modèle **se dégrade** → il faut le ré-entraîner. *Ex. un modèle d'avant-COVID devient faux après.*

💡 **Entretien** : « Ton modèle marche bien aujourd'hui, et dans 6 mois ? » → *Il peut se dégrader à cause du drift (les données évoluent) ; je mets en place un monitoring des performances et je prévois un ré-entraînement régulier.*

12. Glossaire express (à relire la veille)

- **Feature** : variable d'entrée. **Target** : ce qu'on prédit.
- **Supervisé** : avec réponses. **Non supervisé** : sans réponses.
- **Régression** : prédire un nombre. **Classification** : prédire une catégorie.
- **Overfitting** : apprendre par cœur, mauvais sur le nouveau. **Underfitting** : trop simple.
- **Train/Validation/Test** : apprendre / régler / juger.
- **Accuracy / Precision / Recall / F1** : métriques de classification.
- **MAE / RMSE / R²** : métriques de régression.
- **Outlier** : valeur aberrante. **NaN** : valeur manquante.
- **One-Hot / Label encoding** : transformer des catégories en nombres.
- **Normalisation / Standardisation** : mettre à la même échelle.
- **Cross-validation** : valider de façon robuste.
- **Hyperparamètre** : réglage choisi avant l'entraînement.

- **Cluster** : groupe (K-Means). **PCA/ACP** : compresser les colonnes.
 - **Token / Embedding / Transformer / RAG / Hallucination** : monde des LLM.
 - **Data leakage** : le modèle « triche » en voyant des infos qu'il ne devrait pas.
 - **Data drift** : les données changent, le modèle vieillit.
 - **EDA** : exploration des données. **CRISP-DM** : les étapes d'un projet.
-

13. Questions d'entretien technique (avec réponses courtes)

1. **Différence data analyst / data scientist ?** → Analyst = explorer/restituer le présent ; scientist = prédire avec du ML.
2. **Supervisé vs non supervisé ?** → Avec vs sans réponses étiquetées.
3. **Régression vs classification ?** → Prédire un nombre vs une catégorie.
4. **C'est quoi l'overfitting, comment l'éviter ?** → Apprendre par cœur ; éviter via plus de données, modèle plus simple, régularisation, cross-validation.
5. **Train / validation / test ?** → Apprendre / régler les hyperparamètres / juger sur du jamais-vu.
6. **Accuracy peut-elle tromper ?** → Oui, si classes déséquilibrées → utiliser precision/recall/F1.
7. **Precision vs recall ?** → Precision = j'ai raison quand je dis "positif" ; recall = je n'oublie pas de positifs.
8. **Corrélation = causalité ?** → Non (variable cachée, coïncidence) ; il faut une expérimentation.
9. **WHERE vs HAVING ?** → WHERE avant le GROUP BY, HAVING après (sur l'agrégat).
10. **INNER vs LEFT JOIN ?** → INNER = correspondances communes ; LEFT = tout à gauche + NULL à droite.
11. **Gérer les valeurs manquantes ?** → Quantifier, comprendre, puis supprimer ou imputer selon le contexte.
12. **Pourquoi standardiser ?** → Mettre les variables à la même échelle (sauf arbres).
13. **K-Means, c'est quoi ?** → Regrouper en k clusters autour de centres (non supervisé).
14. **C'est quoi un LLM ?** → Un modèle qui prédit le mot suivant, entraîné sur énormément de texte ; sait résumer, traduire, coder ; peut halluciner.
15. **C'est quoi le RAG ?** → Brancher un LLM sur des documents pour qu'il réponde en s'appuyant dessus, sans tout réentraîner.
16. **Une matrice de confusion ?** → Tableau VP/FP/FN/VN pour évaluer une classification.
17. **Random Forest vs arbre seul ?** → Beaucoup d'arbres qui votent = plus robuste, moins d'overfitting.
18. **Ton modèle se dégrade dans le temps, pourquoi ?** → Data drift → monitoring + ré-entraînement.

➡ Pour les réponses longues + la mise en situation, voir aussi ton fichier *Simulation_entretien_data.md*.

14. Plan de révision (checklist avant entretien)

- ☐ **Python/Pandas** : charger un CSV, `head/info/describe`, filtrer, `groupby`, `merge`, gérer les NaN.
- ☐ **SQL** : écrire `SELECT ... GROUP BY ... HAVING`, un `JOIN`, un top N (`ORDER BY ... LIMIT`), connaître WHERE vs HAVING.
- ☐ **Stats** : moyenne/médiane/écart-type, corrélation vs causalité, lire un boxplot.
- ☐ **Process** : savoir dérouler les étapes d'un projet et la mise en situation « explique cette baisse ».
- ☐ **Préparation** : valeurs manquantes, encodage, scaling, train/test split, data leakage.
- ☐ **ML** : supervisé/non supervisé, régression/classification, overfitting, train/val/test, **métriques** (precision/recall/F1, MAE/RMSE/R²), citer 4-5 algos.
- ☐ **Workflow scikit-learn** : savoir le réciter (fit/transform/predict).
- ☐ **Deep learning** : neurones/couches/CNN/RNN/Transformer en 2 phrases.
- ☐ **LLM** : token, embedding, RAG, hallucination ; ton avis nuancé.
- ☐ **MLOps** : déploiement, monitoring, data drift.
- ☐ **Tes 3 projets** : sache les raconter en méthode STAR avec les bonnes notions.

Mantra de l'entretien junior : on n'attend pas que tu saches tout. Sois **clair**, **honnête** (« je ne connais pas, mais voilà comment je m'y prendrais »), et montre ta **logique** et ton **envie d'apprendre**. C'est ça qui fait la différence. Bon courage Lucien ! 💪